

End-to-End n8n Agentic Pipeline Guide

This guide shows how to build and deploy a full **agentic AI pipeline** with n8n, covering both AWS and Hostinger environments. It includes infrastructure setup (reverse proxy, TLS, Docker/Kubernetes, Postgres, Redis, file storage), queue-mode scaling, modular AI providers (local LLMs, OpenAI, Groq), an agentic RAG example with vector DB, Gmail-based email summarization, and best practices (security, monitoring, pruning, backups). Diagrams, code snippets, and configuration blocks illustrate key steps, and all instructions are backed by n8n docs and real-world examples ¹ ² .

1. Infrastructure Setup (AWS & Hostinger)

- **Operating System:** Use a Linux VPS (Ubuntu 24.04 is recommended). Hostinger offers a pre-built "Ubuntu 24.04 with n8n" VPS template (with Docker) and a queue-mode template for easy start ¹ . On AWS, launch an EC2 instance (e.g. Amazon Linux 2023 or Ubuntu) with a proper keypair and security group allowing **SSH, HTTP, and HTTPS** inbound ³ .
- **Docker/Compose or Kubernetes:** For simplicity, use Docker Compose. Install Docker and Docker Compose on your server. Alternatively, for large-scale deployments, consider Amazon EKS (Kubernetes); n8n's AWS docs detail deploying on EKS with Postgres and Ingress ⁴ .
- **Reverse Proxy & TLS:** Deploy a reverse proxy like **Traefik** or **Nginx** to handle HTTPS (Let's Encrypt). For example, run Traefik in Docker: expose ports 80/443 and enable the Docker provider with automatic Let's Encrypt. In Docker Compose you might use:

```
traefik:
  image: traefik:v3.0
  restart: always
  command:
    - "--providers.docker=true"
    - "--entrypoints.web.address=:80"
    - "--entrypoints.websecure.address=:443"
    - "--entrypoints.websecure.http.tls.certresolver=letsencrypt"
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - ./letsencrypt:/letsencrypt
```

This lets Traefik obtain/renew certs automatically. The Medium tutorial shows a similar Traefik setup for EC2 ⁵ ⁶ . On Hostinger, you can likewise install Traefik or Nginx on the VPS and point your domain to the server IP. Hostinger explicitly notes that you *can* configure a reverse proxy (Traefik/Nginx) on their VPS template ⁷ .

- **Database (Postgres):** Use Postgres for persistence. In Docker Compose, run a Postgres container (e.g. `postgres:16-alpine`) with a named volume. For example:

```
postgres:
  image: postgres:16-alpine
  restart: always
  environment:
    - POSTGRES_DB=n8n
    - POSTGRES_USER=n8n
    - POSTGRES_PASSWORD_FILE=/run/secrets/postgres_password
  volumes:
    - postgres_data:/var/lib/postgresql/data
```

Configure the same DB credentials in n8n's environment. The n8n docs recommend Postgres 13+ for production and queue mode (avoid SQLite with queue) ⁸.

- **Cache/Queue (Redis):** Run a Redis container for queue mode. Example:

```
redis:
  image: redis:7-alpine
  restart: always
  volumes:
    - redis_data:/data
```

n8n will connect to Redis via `QUEUE_BULL_REDIS_HOST=redis` etc ⁹. In production, secure Redis as needed (password, firewall).

- **File Storage:** Mount a host volume (e.g. `/home/node/.n8n`) for n8n's files and custom data. If using queue mode and storing binary data, consider external storage: the docs warn that *filesystem* binary storage isn't supported in queue mode and recommend using S3 ¹⁰. You can use AWS S3 or a similar bucket for attachments (configure `N8N_FILES_*` env vars or MinIO). For certificates and secrets, use Docker secrets or Vault.

Infrastructure Note: On AWS you might also use Route 53 for DNS to point to your instance, or use an Elastic Load Balancer/ALB with an HTTPS listener (especially if using EKS). For simplicity, the EC2 method (security group + Traefik) works well ³ ⁵. Hostinger's stack is entirely on the VPS itself, managed via hPanel or CLI.

2. Queue Mode & Worker Scaling

Figure: n8n queue mode architecture. A single "main" instance enqueues workflow executions to Redis, and multiple worker instances pick up jobs from Redis and execute them ² ¹¹.

Running in **queue mode** is essential for production-scale workflows ¹². In this mode, one n8n instance (the *main*) handles all triggers and webhooks and quickly enqueues tasks; multiple *worker* instances then perform the actual workflow runs. As n8n docs explain, the main node generates execution IDs and pushes them to Redis, and any available worker can pull a job, fetch the workflow from the database, execute it,

and write the results back ². This allows you to **scale horizontally** by adding more worker containers or pods as load increases.

Key steps to enable queue mode:

- **Enable queue:** Set environment variable `EXECUTIONS_MODE=queue` on both main and worker processes ⁹.
- **Shared Encryption Key:** If you use encrypted credentials, ensure all workers and the main share the same `N8N_ENCRYPTION_KEY` (Toolsana mounts it from a Docker secret) ¹³.
- **Redis Configuration:** Configure Redis host/port via `QUEUE_BULL_REDIS_HOST=redis` (or the hostname/IP of your Redis server) ¹⁴. Ensure `redis` is reachable and healthy.
- **Workers:** Start one or more worker processes, either on the same machine or others. In Docker Compose, you can define an `n8n-worker` service (same image) with `command: ["worker"]`. For example:

```
n8n-worker:
  <<: *shared_n8n_config
  command: ["n8n", "worker", "--concurrency=5"]
  environment:
    - N8N_CONCURRENCY_PRODUCTION_LIMIT=5
    - N8N_GRACEFUL_SHUTDOWN_TIMEOUT=30
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis
```

Toolsana's example runs workers with a concurrency flag and multiple replicas (e.g. 2 replicas at 10 concurrency each) ¹¹.

- **Worker Scaling:** You can scale workers by running more containers or pods. For Docker Swarm/ECS/K8s, set replicas appropriately. Monitor CPU usage and Redis queue length to decide scaling. The Toolsana guide recommends watching queue lengths (via `redis-cli LLEN bull:n8n:wait`) and using Prometheus/Grafana to visualize metrics ¹⁵ ¹⁶.

Redis is the queue broker in this setup. The lifecycle is: main → Redis (enqueue) → worker → DB update → Redis (notification) → main sees completion ². The diagram above illustrates this flow. Once configured, you can add or remove workers dynamically without interrupting the main process.

3. Modular AI Integration (Local & Cloud LLMs)

n8n supports flexible AI providers. You can plug in **local models** or cloud APIs and even switch between them in a workflow.

- **Local LLMs (Ollama, llama.cpp, etc.):** For on-prem inference, n8n can integrate with tools like Ollama or llama.cpp if they expose an HTTP endpoint or CLI. For example, the n8n team's *Self-Hosted AI Starter Kit* bundles n8n with **Ollama** and **Qdrant** in Docker Compose ¹⁷. Ollama can serve local models (e.g. Llama2, etc.) via its HTTP API, and n8n's LLMs nodes (like the **Ollama Model** node) can call it. In practice, install Ollama on the same host or container and use it as an API endpoint. (Similarly, llama.cpp can be served via a small web service to accept prompts.)

- **OpenAI/GPT:** n8n has built-in OpenAI nodes for ChatGPT/GPT models. Simply add the **OpenAI** credential with your API key, then use the OpenAI node in your flow to ask questions. This lets you call GPT-3.5/4 or ChatGPT easily. The n8n integrations page highlights that you can connect OpenAI with 1000+ apps seamlessly ¹⁸ (conceptually).
- **Groq:** For Groq's LLM API, n8n provides a **Groq Chat Model** node. You create an API key in Groq's console (under **API Keys**) and paste it into n8n's Groq credential ¹⁹. Then the Groq node can be used like any other LLM node.
- **Switching Providers:** In your workflow, you can route prompts to different providers. For example, use a Switch or IF node on some variable (e.g. an incoming request's field) and conditionally call the OpenAI node or the Ollama node. Environment variables for API keys or endpoints should be stored in n8n's credentials (not hard-coded).

This modular setup means the **same agent logic** can flexibly call *any* LLM. For instance, you might first try your fast local model (Ollama) and fall back to OpenAI if needed. n8n flows can dynamically decide which "tool" to use, as shown in agentic workflows below ²⁰ ²¹.

4. Agentic RAG Pipeline Example

An **Agentic Retrieval-Augmented Generation (RAG)** pipeline uses an AI agent node that chooses tools (like a vector DB or a search engine) dynamically, then verifies its answer. Here's an outline example:

- **Indexing & Storage:** First, index your knowledge base (PDFs, docs) into a vector database. For example, use the **Weaviate Vector Store** node. Set it to *Insert Documents* mode, specify your Weaviate URL/collection, and provide the content field (e.g. text or summary) as the embedding source ²². The node will embed the text (with an embedding model) and store vectors in Weaviate.
- **Agent Tools:** Configure an **AI Agent** node with multiple tools. For example, add Weaviate as one tool (through its LangChain integration) and an external search API as another. As Mihai Farcas describes, the AI Agent node can be connected to a RAG vector store and a web search tool; at runtime the agent "analyzes the query and decides which tool is best suited" ²⁰. If the user's question is factoid-based, route to Weaviate; if it needs current info, use the search tool.
- **Workflow Flow:** A workflow could look like: HTTP Trigger → AI Agent Node → (Weaviate/Search as tools) → AI Agent outputs answer. Internally, the agent will loop: retrieve relevant context (via the chosen tool), then generate an answer with its LLM (with context). You can also add a *verification step*: for instance, after the agent answers, you could have another OpenAI node ask "Is the above answer supported by facts?" to double-check (this is an example of an agent "critiquing" itself).
- **Vector DB:** Weaviate (self-hosted or cloud) is fully supported as shown in n8n's docs. Its Vector Store node "can insert or retrieve documents... and connect directly to an AI agent to use as a tool" ²³. (Other vector DBs like Qdrant/Pinecone are similarly available via LangChain integration.) If using local FAISS instead, you'd run FAISS in a Python service and call it via n8n's HTTP or Code node.
- **Agentic Logic:** The agentic approach is "dynamic, adaptive, autonomous" ²⁴. Unlike simple RAG (retrieve-then-answer), the agent can decide the retrieval strategy and even "critique its own response for completeness" ²¹. By combining a vector store tool and a search tool, the AI agent workflow gains the full power of Agentic RAG ²⁰ ²¹.

Tip: n8n has detailed AI agent examples. The Weaviate blog shows using an AI Agent to identify research trends by querying a Weaviate-powered RAG system ²⁵. Use these as inspiration and adapt to your domain.

5. Gmail Email Summarization Agent

To automate email tasks, n8n's Gmail nodes let the AI read, summarize, and reply to emails. For example:

- **Trigger:** Use **Gmail Trigger** node to poll for new emails. Set it to "Message Received" mode. It returns message IDs and headers (From, Subject, etc.) ²⁶.
- **Fetch Email:** Chain a **Gmail** node (Get Message) using the ID from the trigger to retrieve the full email body.
- **AI Summarize:** Pass the email text into an AI node (e.g. OpenAI or an AI Agent) to generate a summary or action items. For instance, an AI Agent node could be configured with a prompt like "Summarize the key points and tasks from this email: {{ \$json["body"] }}" and output a structured response.
- **Optional Verification:** You might run a second check by asking the agent to verify consistency (e.g. "Are the tasks covered by the summary?").
- **Reply:** Use the **Gmail** node's **Reply** operation to send an email back (to the original sender or others). The AI-generated text becomes the reply body.

A concrete template is the "Morning Mail Summary Agent" example. It triggers daily, fetches recent emails, then an AI agent produces a structured report of *Key updates, Issues, and Action items*, which is emailed to the team ²⁷. The key steps there are: schedule trigger → fetch Gmail messages → aggregate metadata → AI summarize → send digest ²⁷. You could adapt this to continuously process incoming email: for each new message, trigger, summarize (via AI), and auto-reply or forward as needed.

Key point: n8n's Gmail node supports reply/send operations (e.g. "Reply to a message") ²⁸. Combine that with AI nodes to automate email workflows end-to-end.

6. Example Workflows & Templates

n8n workflows are JSON that can be imported. To get started, use community templates:

- **Email Summary Example:** The "Daily Email Digest with AI Summarization" template (n8n.io) includes a 6-node workflow that does exactly the steps above ²⁷. You can download its JSON and import into your n8n instance. It shows how to connect a Gmail Trigger, a Loop to aggregate, an AI chain (LangChain), and Gmail send.
- **Agentic RAG Example:** Look for tutorials on "AI Agent workflow" on n8n's docs or blog. These often provide sample flows (e.g. PDF ingestion into Weaviate, then agent query). You can export/import those JSON files.
- **Docker Compose Stack:** Use a `docker-compose.yml` template for the full stack. For instance, combining the examples above, your compose might include:

```
version: '3.8'
services:
  traefik:
    image: traefik:v3.0
    restart: always
```

```

command:
  - "--providers.docker=true"
  - "--entrypoints.web.address=:80"
  - "--entrypoints.websecure.address=:443"
  - "--entrypoints.websecure.http.tls.certresolver=letsencrypt"
ports:
  - "80:80"
  - "443:443"
volumes:
  - /var/run/docker.sock:/var/run/docker.sock:ro
  - ./letsencrypt:/letsencrypt

n8n-main:
  image: n8nio/n8n:latest
  restart: always
  depends_on:
    - postgres
    - redis
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.n8n.rule=Host(`n8n.example.com`)"
    - "traefik.http.routers.n8n.tls.certresolver=letsencrypt"
    - "traefik.http.routers.n8n.entrypoints=websecure"
  environment:
    - DB_TYPE=postgresdb
    - DB_POSTGRESDB_HOST=postgres
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis
    - N8N_HOST=n8n.example.com
    - N8N_PROTOCOL=https
    - WEBHOOK_URL=https://n8n.example.com/
    - N8N_ENCRYPTION_KEY_FILE=/run/secrets/encryption_key
    - N8N_SECURE_COOKIE=true
    - N8N_SAMESITE_COOKIE=strict

n8n-worker:
  image: n8nio/n8n:latest
  restart: always
  depends_on:
    - redis
    - postgres
  command: ["n8n", "worker"]
  environment:
    - EXECUTIONS_MODE=queue
    - QUEUE_BULL_REDIS_HOST=redis

postgres:
  image: postgres:16-alpine

```

```

restart: always
environment:
  - POSTGRES_DB=n8n
  - POSTGRES_USER=n8n
  - POSTGRES_PASSWORD_FILE=/run/secrets/postgres_password
volumes:
  - postgres_data:/var/lib/postgresql/data

redis:
  image: redis:7-alpine
  restart: always
  volumes:
    - redis_data:/data

volumes:
  postgres_data:
  redis_data:

```

This example (based on Toolsana and other guides) shows main and worker services with queue mode environment and Traefik labels ²⁹ ⁶ . Customize domains, secrets, and replicas as needed.

- **Kubernetes Manifests:** If using EKS/K8s, clone the n8n-hosting repo (github.com/n8n-io/n8n-hosting) and follow the AWS guide. It provides YAML for Deployments, Services, Ingress with TLS, and PersistentVolumes for Postgres ³⁰ .

7. Security, Monitoring, and Maintenance

- **Authentication & Credentials:** Always run n8n in secure mode. Set an encryption key (`N8N_ENCRYPTION_KEY`) and enable secure cookies (`N8N_SECURE_COOKIE=true` , `N8N_SAMESITE_COOKIE=strict`) ¹³ . Store credentials (API keys, OAuth tokens) in n8n's credential manager, not in plain environment. Use Docker secrets or a vault for sensitive values (Toolsana mounts the encryption key from a secret) ¹³ . For web UI, consider enabling Basic Auth or using OAuth/SSO (n8n Enterprise) to restrict access.
- **Network Security:** Only expose the necessary ports (80/443 to Traefik). The n8n main container should ideally bind only to localhost (as in our examples) and rely on Traefik for external access. Hostinger's VPS can run a firewall (iptables or equivalent) to block unused ports.
- **Monitoring:** Track n8n's health and performance. Toolsana suggests monitoring queue lengths and worker load ¹⁵ . You can enable n8n's Prometheus metrics by setting `--tunnel` or using the built-in `/metrics` endpoint. A Prometheus job can scrape n8n at `/metrics` (see Toolsana snippet) ³¹ . Likewise scrape Postgres and Redis metrics. Use Grafana to visualize. Set alerts on high queue wait lengths or failed executions.
- **Pruning Data:** Prevent the database from growing unchecked. Enable n8n's auto-pruning:

```
EXECUTIONS_DATA_PRUNE=true
EXECUTIONS_DATA_PRUNE_MAX_COUNT=10000
EXECUTIONS_DATA_MAX_AGE=168
```

These will delete old execution records beyond 1 week or 10,000 entries ³². Also periodically remove old n8n files (credentials JSON, log files, etc.) if you store them locally.

- **Backups:** Implement backups for disaster recovery. A robust strategy (Toolsana example) is:
- **Dump the database:** e.g. `pg_dump` the n8n Postgres into a timestamped file ³³.
- **Backup n8n data:** tarball the `/home/node/.n8n` (workflows, credentials) ³⁴.
- **Export Workflows/Credentials:** Use `n8n export:workflow --output=...` and same for credentials via the n8n CLI inside the container ³⁵.
- **Rotate:** Keep, say, 30 days of backups and delete older ones ³⁶. Optionally upload to S3 or offsite ³⁷.
- **Scripts & Cron:** Write a shell script to do the above (the Toolsana sample is ~40 lines) and run it daily via cron ³⁸ ³⁹.
- **Performance Tuning:** Based on metrics (CPU, queue wait, DB size), tune concurrency and resource limits. For example, increase `N8N_CONCURRENCY_PRODUCTION_LIMIT` to allow workers to run more jobs in parallel, or allocate more RAM (`NODE_OPTIONS=--max-old-space-size`) ⁴⁰ ¹⁶. The Toolsana guide suggests analyzing Grafana dashboards to find bottlenecks and adjust worker count or memory accordingly.

Keeping all components updated (Docker images, OS packages) and regularly reviewing logs completes a sound maintenance plan.

8. Deployment & Scaling on AWS and Hostinger

- **AWS (EC2):** Spin up an EC2 instance and follow the Docker setup above. Ensure your domain's DNS (Route 53 or other) points to the instance's public IP. Use the Traefik or Nginx proxy approach for TLS. For high availability, you could run multiple EC2s behind a load balancer. For ultimate scalability, consider EKS: n8n's AWS guide uses `eksctl` to create a cluster and applies Kubernetes manifests for n8n, Postgres (with a PVC using gp3), Redis, and an Ingress with TLS ⁴. EKS allows you to autoscale pods and nodes as needed.
- **Hostinger VPS:** Use the "n8n VPS hosting" package (Linux VPS). You can deploy via their UI or SSH. The prebuilt Ubuntu template comes with Docker ready. You can import your Docker Compose stack or use their one-click n8n image. Hostinger's docs suggest the queue-mode template for scaling ¹. Point your domain to the VPS IP in hPanel. You can set up Nginx or Traefik on the VPS (install with apt or Docker) and obtain a Let's Encrypt cert (Certbot) or use Traefik auto-HTTPS as above ⁷. Hostinger support can assist with VPS networking. All otherwise applies as above: run Postgres/Redis containers, mount volumes, configure env vars, etc.
- **Scaling:** On AWS, autoscale the EC2 or use managed DB. On Hostinger, you may need to manually resize the VPS plan (they have 1-8 CPU plans ⁴¹) as load grows. In both cases, adding more n8n workers (even on multiple machines) can be done by point them at the same Redis/Postgres. K8s (on AWS) is inherently scalable via pod replicas.

Follow these steps and configurations to deploy a robust n8n-based agentic pipeline: set up secure infrastructure with Docker/compose or Kubernetes, enable queue mode with Redis and multiple workers ² ¹¹, integrate AI models modularly (local Ollama or cloud GPT/Groq) ¹⁷ ¹⁹, build workflows with RAG and agent logic (using Weaviate or FAISS for vector search) ²³ ²⁰, and include automation like email summarization (Gmail trigger → AI → Gmail reply) ²⁶ ²⁷. Use n8n's templates and the example code snippets above as starting points. By following security, monitoring, and backup best practices (encryption keys, pruning, automated backups) ²⁹ ³⁴, you will have a production-ready agentic AI automation platform on either AWS or Hostinger.

Sources: Official n8n documentation and community guides were used extensively ² ²⁰ ²⁷ ²⁹. These provide detailed instructions on queue mode, AI agents, Gmail integration, and Docker/Kubernetes deployment. The Hostinger and Medium articles illustrate practical deployment examples ¹ ⁶. All code examples and recommendations above are derived from these sources.

¹ ⁷ ⁴¹ Self-hosted n8n | Secure and scalable automation

<https://www.hostinger.com/self-hosted-n8n>

² ⁸ ⁹ ¹⁰ ¹² Configuring queue mode | n8n Docs

<https://docs.n8n.io/hosting/scaling/queue-mode/>

³ ⁵ ⁶ How to Deploy n8n to AWS. Part I : EC2 | by Destiya Wijayanto | Medium

<https://medium.com/@destiya.dian/how-to-deploy-n8n-to-aws-part-i-ec2-c3a7da3074ec>

⁴ ³⁰ Amazon Web Services | n8n Docs

<https://docs.n8n.io/hosting/installation/server-setups/aws/>

¹¹ ¹³ ¹⁴ ¹⁵ ¹⁶ ²⁹ ³¹ ³² ³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁸ ³⁹ ⁴⁰ Production n8n Docker Compose Setup with PostgreSQL & Redis: Complete Guide | 2025

<https://toolsana.com/blog/n8n-docker-compose-production-postgresql-redis/>

¹⁷ How to Run a Local LLM: Complete Guide to Setup & Best Models (2025) – n8n Blog

<https://blog.n8n.io/local-llm/>

¹⁸ OpenAI integrations | Workflow automation with n8n

<https://n8n.io/integrations/openai/>

¹⁹ Groq credentials | n8n Docs

<https://docs.n8n.io/integrations/builtin/credentials/groq/>

²⁰ ²¹ ²⁴ Agentic RAG: A Guide to Building Autonomous AI Systems – n8n Blog

<https://blog.n8n.io/agentic-rag/>

²² ²⁵ AI Agent Workflow Automation with n8n and Weaviate | Weaviate

<https://weaviate.io/blog/agent-workflow-automation-n8n-weaviate>

²³ Weaviate Vector Store node documentation | n8n Docs

<https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.vectorstoreweaviate/>

²⁶ Gmail Trigger node documentation | n8n Docs

<https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.gmailtrigger/>

²⁷ **Daily Email Digest with AI Summarization using Gmail, OpenRouter and LangChain | n8n workflow template**

<https://n8n.io/workflows/5003-daily-email-digest-with-ai-summarization-using-gmail-openrouter-and-langchain/>

²⁸ **Gmail node documentation | n8n Docs**

<https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.gmail/>